ChatGPT said:

The line is effectively:

```
C++

ElementAccum lse_logsum =
    (lse_sum == 0.f || lse_sum != lse_sum)
        ? INFINITY
        : logf(lse_sum) + lse_max;
```

Here `lse_sum != lse_sum` is the NaN test. A NaN is the one IEEE floating-point value that is not equal to itself:

```
C++

3.0f == 3.0f     // true
NAN  == NAN      // false
NAN  != NAN      // true
```

`lse_sum` is the stabilized sum in a log-sum-exp combination across attention splits:

$$\text{lse_sum} = \sum_j e^{\text{LSE}_j - \text{lse_max}}.$$

Normally it should be positive and finite, so the combined log-sum-exp is:

$$\log(\text{lse_sum}) + \text{lse_max}.$$

The special branch deals with two bad/degenerate cases:

1. `lse_sum == 0`
   Every split supplied `LSE = -∞`, meaning this query row has no valid keys—for example, it is fully masked. Mathematically, its final LSE should be $-\infty$.
2. `lse_sum` is NaN
   Some upstream LSE/arithmetic was already invalid, such as a NaN score, or an invalid $+\infty - +\infty$ subtraction. `logf(NaN)` would remain NaN, so the code avoids storing a NaN LSE value.

Why choose `+INFINITY` as the sentinel rather than the mathematically expected `-INFINITY`? The next code computes the per-split normalization scale:

```
C++

sLSE[row][col] = expf(lse_accum(l) - lse_logsum);
```

For an empty row, every `lse_accum(l)` is `-INFINITY`.

- If it used `lse_logsum = -INFINITY`, it would calculate `-INFINITY - -INFINITY`, which is NaN; then `expf(NaN)` is NaN.
- With `lse_logsum = +INFINITY`, it calculates `-INFINITY - +INFINITY = -INFINITY`; then `expf(-INFINITY) = 0`.

So the sentinel forces all split weights to zero and avoids manufacturing NaNs from a fully masked row.

The NaN part has a narrower purpose: it prevents the **combined LSE output itself** from becoming NaN. It does not generally “repair” a NaN in `lse_accum`; such a NaN will still propagate into the later `expf(lse_accum(l) - lse_logsum)` for that particular split. The code is chiefly guarding the normal reduction/empty-row path, not serving as a complete NaN sanitizer. GitLab